Master's Thesis Proposal and System Architecture Document

LM-IDNet: An Adaptive Dirichlet-Multinomial System for Unsupervised IoT Network Anomaly Detection

Archived design/experiment record. This document retains its original implementation boundary. The agreed detector and final-partition evaluation are complete; current results and supported interfaces are documented in `docs/lm_idnet_full_project_report.tex` and `reports/final_evaluation/results.json`. Unselected proposals are future work, as collected in `docs/future-work.md`, rather than an active software backlog. Historical timings and findings below retain their original meaning.

Contents

Purpose of This Document **3**

1 Introduction and Problem Statement **3**
- 1.1 The Real-World Problem . . . 3
- 1.2 The Statistical Problem Hidden Inside IoT Traffic . . . 4
- 1.3 Why Standard Models Are Not Enough . . . 4
- 1.4 Thesis Objective . . . 4
- 1.5 Research Questions . . . 5
- 1.6 Expected Contributions . . . 5

2 Mathematical and Theoretical Foundations **5**
- 2.1 From Network Packets to Count Vectors . . . 5
- 2.2 Categorical and Multinomial Distributions . . . 6
- 2.3 Overdispersion . . . 7
- 2.4 The Dirichlet Distribution . . . 7
- 2.5 The Dirichlet-Multinomial Distribution . . . 8
- 2.6 Mean-Precision and Overdispersion Parameterization . . . 8
- 2.7 Maximum Likelihood Estimation . . . 9
- 2.8 Digamma Function and Fixed-Point Estimation . . . 9
- 2.9 The Numerical Problem: Paired Log-Gamma Differences . . . 9
- 2.10 Languasco-Migliardi Log-Likelihood Computation . . . 10
- 2.11 Likelihood as an Anomaly Score . . . 10
- 2.12 Topic Modeling Concepts as an Intuition Bridge . . . 10

3 Detailed Literature Review **11**
- 3.1 Project Brief: Research Proposals Based on the Dirichlet-Multinomial . . . 11
- 3.2 Minka: Estimating a Dirichlet Distribution . . . 12
- 3.3 Yu and Shaw: Efficient Computation of the Dirichlet-Multinomial Log-Likelihood 12
- 3.4 Languasco and Migliardi: Fast Computation of the Dirichlet-Multinomial Log-Likelihood . . . 13

3.5 Efficient Analysis of Overdispersed Data Using Accurate Dirichlet-Multinomial Computation . . . 13
3.6 ALM IoT-CR: Overdispersed IoT Count Modelling . . . 14
3.7 ALM2: Fast and Accurate Implementation of the Dirichlet-Multinomial Log-Likelihood . . . 15
3.8 Nespoli et al.: Optimal Countermeasure Selection Against Cyber Attacks . . . 15
3.9 Motivating Note on Why ALM Matters Beyond This Thesis . . . 16
3.10 Synthesis and Research Gap . . . 16

4 Proposed System Architecture and Process Description 17
4.1 System Name and High-Level Concept . . . 17
4.2 Architecture Overview . . . 17
4.3 Stage 1: Data Ingestion . . . 18
4.4 Stage 2: Window Aggregation . . . 18
4.5 Stage 3: Data Validation and Diagnostics . . . 18
4.6 Stage 4: Training the Normal Model . . . 19
4.7 Stage 5: Likelihood Engine . . . 20
4.8 Stage 6: Threshold Calibration . . . 20
4.9 Stage 7: Online Anomaly Detection . . . 20
4.10 Stage 8: Explanation and Category Contributions . . . 21
4.11 Stage 9: Adaptive Retraining . . . 21
4.12 Stage 11: Reaction Support . . . 22
4.13 Current Repository Mapping . . . 22
4.14 Evaluation Design . . . 23
4.15 Baselines . . . 23

5 Roadmap and Implementation Strategy 24
5.1 Phase 0: Research Setup and Reproducibility . . . 24
5.2 Phase 1: Data Processing Hardening . . . 24
5.3 Phase 2: Dirichlet-Multinomial Modeling . . . 24
5.4 Phase 3: Likelihood Backend Abstraction . . . 25
5.5 Phase 4: Threshold Calibration and Anomaly Scoring . . . 25
5.6 Phase 5: Experimental Anomaly Evaluation . . . 25
5.7 Phase 6: Adaptive Retraining . . . 26
5.8 Phase 8: Edge Deployment Study . . . 26
5.9 Possible Theoretical Extensions . . . 26
5.10 Suggested Thesis Chapter Structure . . . 27
5.11 Final Expected Artifacts . . . 27

Source Map 27

Concluding Statement 28

Purpose of This Document

This document proposes a complete Master's thesis project and a corresponding system architecture for unsupervised anomaly detection in IoT network traffic. It is written as a standalone guide for a reader who may know basic programming and statistics, but has no prior knowledge of Dirichlet-Multinomial modeling, overdispersed count data, probabilistic topic models, or IoT anomaly detection.

The proposal is based on the project brief in `resouces/Proposte_di_Ricerca_basate_su_Dirichlet_Multinomial.pdf` and on the supporting academic papers in the same folder. The brief contains two possible research directions. The direction developed here is the second project in the brief: **LM-IDNet, an adaptive anomaly detection system for IoT and cybersecurity based on online estimation of Dirichlet-Multinomial parameters using the Languasco-Migliardi method**. The first project, a Dirichlet-Softmax layer for neural networks, is treated as a possible extension rather than as the central system, because the repository and datasets are already structured around IoT traffic modeling.

The document has five required sections:

1. Introduction and problem statement.
2. Mathematical and theoretical foundations.
3. Detailed literature review.
4. Proposed system architecture and process description.
5. Roadmap and implementation strategy.

1 Introduction and Problem Statement

1.1 The Real-World Problem

The Internet of Things has moved networked computation into homes, factories, hospitals, transportation systems, offices, energy systems, and public infrastructure. Many IoT devices are small, cheap, and continuously connected: smart cameras, thermostats, light switches, voice assistants, wearable sensors, industrial monitors, and embedded controllers. These devices often have limited memory, limited CPU capacity, long deployment lifetimes, and weak update practices. At the same time, they continuously generate network traffic and interact with local networks and external cloud services.

This creates a cybersecurity problem with several properties:

- IoT devices are attractive attack targets because they are numerous and often poorly maintained.
- A compromised IoT device can leak private information, participate in botnets, scan the local network, attack other devices, or disrupt critical services.
- Signature-based defenses are insufficient because new attacks, misconfigurations, and zero-day vulnerabilities may not match known signatures.
- Many IoT environments cannot support heavy deep-learning detection systems on-device or near the edge.
- Network behavior is not constant; normal behavior can drift because of firmware updates, user behavior, cloud-service changes, time of day, and device lifecycle changes.

Therefore, a useful IoT anomaly detection system should be lightweight, interpretable, adaptive, and able to model "normal" behavior without requiring labeled attack data. This motivates an unsupervised statistical approach: learn the normal traffic distribution of a device, score future traffic against that learned distribution, and raise alerts when observed behavior becomes statistically unlikely.

1.2 The Statistical Problem Hidden Inside IoT Traffic

The thesis brief proposes a system that models IoT traffic as categorical count data. For example, in a 10-minute time window, a smart camera may produce a vector such as:

$$x_t = \begin{bmatrix} \text{TCP_count} & \text{UDP_count} & \text{SSDP_count} & \text{ARP_count} \end{bmatrix}.$$

Each window is not a single scalar measurement. It is a vector of counts across multiple traffic categories. The total number of packets in the window may vary, and the composition of traffic may vary as well.

The core statistical task is:

> Given historical count vectors that represent normal IoT traffic, estimate a probability model of normal behavior and use the log-likelihood of new count vectors as an anomaly score.

A high log-likelihood means the new window resembles normal behavior. A very low log-likelihood means the window is unusual under the learned model. This is useful because many attacks and failures change traffic composition: for example, more scanning traffic, unusual UDP bursts, abnormal ARP activity, cloud communication failures, or discovery-protocol abuse.

1.3 Why Standard Models Are Not Enough

A first attempt might use simple distributions:

- A Gaussian model for continuous features.
- Independent Poisson models for packet counts.
- A Multinomial model for categorical packet composition.

These are not sufficient in the setting targeted by the project brief.

The Gaussian distribution is natural for continuous measurements with symmetric variation, but packet counts are non-negative integers and often skewed. A Gaussian model can assign probability to impossible negative counts and may poorly represent bursts.

The Poisson distribution is natural for counts, but it assumes that the mean and variance are equal. Real network traffic often violates this assumption. It may have long quiet periods, sudden bursts, periodic maintenance, cloud-service synchronization, discovery traffic, and user-driven activity. The variance can be much larger than the mean. This phenomenon is called **overdispersion**.

The Multinomial distribution is natural for categorical counts when the total number of trials is fixed and the category probabilities are fixed. However, it assumes that all packets in a window are conditionally independent draws from one fixed probability vector. IoT traffic often violates this assumption because packet types are correlated. For example, a camera's TCP traffic may rise together with certain UDP discovery patterns, or ARP activity may occur in bursts when network state changes.

The Dirichlet-Multinomial distribution directly addresses these limitations. It models categorical counts while allowing the underlying category probabilities themselves to vary from window to window. This extra variation is exactly what makes the model appropriate for overdispersed multicategory count data.

1.4 Thesis Objective

The proposed thesis objective is:

> Design, implement, and evaluate an adaptive, unsupervised anomaly detection system for IoT network traffic using a Dirichlet-Multinomial normal-behavior model, with

efficient and accurate log-likelihood computation based on the Languasco-Migliardi method.

The system should:

- Ingest packet traces or NetFlow-like summaries from an IoT device.
- Aggregate traffic into fixed-length count windows.
- Estimate the Dirichlet-Multinomial parameters of normal traffic.
- Score new windows by log-likelihood.
- Detect anomalies using calibrated likelihood thresholds.
- Adapt the normal model over time while reducing poisoning risk.
- Support lightweight edge deployment.
- Produce interpretable evidence about which traffic categories contributed to the anomaly.
- Evaluate accuracy, false positive rate, true positive rate, likelihood distributions, and runtime.

1.5 Research Questions

The thesis can be organized around the following research questions:

RQ1. Modeling adequacy: Does the Dirichlet-Multinomial distribution model normal IoT traffic windows better than simpler count models such as independent Poisson or Multinomial baselines?

RQ2. Anomaly detection: Can Dirichlet-Multinomial log-likelihood scoring detect anomalous IoT traffic with acceptable true positive and false positive rates?

RQ3. Numerical reliability: Does the Languasco-Migliardi log-likelihood computation improve stability and runtime compared with standard log-gamma implementations and the Yu-Shaw method, especially under overdispersion?

RQ4. Adaptation: Can the system update its normal profile over time without absorbing anomalous behavior into the normal model?

RQ5. Edge suitability: Can the full pipeline run within realistic edge constraints on CPU, memory, and latency?

1.6 Expected Contributions

The thesis should aim to produce the following contributions:

- A clear statistical formulation of IoT anomaly detection as overdispersed categorical count modeling.
- A working implementation of a Dirichlet-Multinomial anomaly detector using fixed-time traffic windows.
- An efficient likelihood backend based on the Languasco-Migliardi method, or a clean abstraction that allows this backend to replace standard log-gamma computation.
- A threshold-calibrated anomaly scoring method.
- An adaptive retraining strategy with guardrails against model poisoning.
- A reproducible experimental evaluation on smart-camera traffic.
- A system architecture suitable for extension to NetFlow data and edge deployment.

2 Mathematical and Theoretical Foundations

2.1 From Network Packets to Count Vectors

The system begins with packet-level network observations. A raw packet trace is too detailed to model directly in a lightweight edge detector. Instead, the trace is aggregated into time windows.

Let time be divided into windows of fixed duration, for example 10 minutes. In each window, packets are assigned to one of K traffic categories. In the current repository, the default categories are:

$$\text{TCP}, \quad \text{UDP}, \quad \text{SSDP}, \quad \text{ARP}.$$

For a given window t, define:

$$\mathbf{x}_t = (x_{t1}, x_{t2}, \ldots, x_{tK}),$$

where x_{tk} is the number of packets in category k during window t.

The total packet count in the window is:

$$N_t = \sum_{k=1}^{K} x_{tk}.$$

The empirical category frequencies are:

$$f_{tk} = \frac{x_{tk}}{N_t},$$

when $N_t > 0$. Silent windows, where $N_t = 0$, must be handled carefully. They can be retained as meaningful "no traffic" observations, modeled separately, or excluded from likelihood scoring depending on the experimental design.

2.2 Categorical and Multinomial Distributions

The simplest model for categorical observations is the Categorical distribution. If one packet belongs to one of K categories, and the probability of category k is p_k, then:

$$\Pr(\text{category} = k) = p_k,$$

where:

$$p_k \geq 0, \qquad \sum_{k=1}^{K} p_k = 1.$$

The vector $\mathbf{p} = (p_1, \ldots, p_K)$ lies on the probability simplex.

If a window contains N packets and each packet is independently assigned to a category with fixed probability vector $\mathbf{p}$, then the count vector $\mathbf{x} = (x_1, \ldots, x_K)$ follows a Multinomial distribution:

$$\Pr(\mathbf{x} \mid N, \mathbf{p}) = \frac{N!}{x_1! x_2! \cdots x_K!} \prod_{k=1}^{K} p_k^{x_k}.$$

The Multinomial distribution is useful, but it has a strong assumption: the probability vector $\mathbf{p}$ is fixed. If traffic composition changes from window to window because of normal device variability, the Multinomial model underestimates variability. This creates too many false positives because normal bursts look statistically impossible.

2.3 Overdispersion

Overdispersion means that the observed variance is larger than the variance expected under a chosen model.

For a Poisson count model, the variance should equal the mean:

$$\mathrm{Var}(X) = \mathbb{E}[X].$$

If the observed variance is much larger than the mean, the data are overdispersed relative to the Poisson model.

For Multinomial data, overdispersion means that count proportions vary more across windows than a fixed-probability Multinomial model predicts. In network traffic, this is common. Even during normal behavior, an IoT device may move among several modes:

- Idle.
- Local discovery.
- Cloud upload.
- Video streaming.
- Firmware update.
- DNS or control-plane communication.

Each mode has a different traffic mixture. A single fixed $\mathbf{p}$ is therefore too rigid.

2.4 The Dirichlet Distribution

The Dirichlet distribution is a probability distribution over probability vectors.

This is an important conceptual step:

- A Multinomial distribution uses a probability vector $\mathbf{p}$.
- A Dirichlet distribution models uncertainty and variability in $\mathbf{p}$ itself.

Let:

$$\mathbf{p} = (p_1, \ldots, p_K),$$

where $p_k \geq 0$ and $\sum_k p_k = 1$. A Dirichlet distribution with parameter vector:

$$\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_K)$$

has density:

$$\mathrm{Dirichlet}(\mathbf{p} \mid \boldsymbol{\alpha}) = \frac{\Gamma(\alpha_0)}{\prod_{k=1}^{K} \Gamma(\alpha_k)} \prod_{k=1}^{K} p_k^{\alpha_k - 1},$$

where:

$$\alpha_0 = \sum_{k=1}^{K} \alpha_k.$$

The parameters have intuitive meanings:

- α_k controls the tendency toward category k.
- α_0 is the total concentration or precision.
- The mean probability of category k is:

$$\mathbb{E}[p_k] = \frac{\alpha_k}{\alpha_0}.$$

If α_0 is large, probability vectors sampled from the Dirichlet are tightly concentrated around the mean. If α_0 is small, sampled probability vectors vary widely. This is why the Dirichlet is useful for overdispersed data: it lets the category probabilities fluctuate between windows.

2.5 The Dirichlet-Multinomial Distribution

The Dirichlet-Multinomial distribution is a compound distribution. It describes the following generative process:

1. Draw a window-specific probability vector:

$$\mathbf{p}_t \sim \text{Dirichlet}(\boldsymbol{\alpha}).$$

2. Draw the count vector for the window:

$$\mathbf{x}_t \sim \text{Multinomial}(N_t, \mathbf{p}_t).$$

Because $\mathbf{p}_t$ changes from one window to another, the resulting count vectors show more variability than ordinary Multinomial counts. This makes the Dirichlet-Multinomial a natural model for IoT traffic windows.

The probability mass function, omitting only notation details, is:

$$\Pr(\mathbf{x} \mid \boldsymbol{\alpha}) = \frac{N!}{\prod_{k=1}^{K} x_k!} \frac{\Gamma(\alpha_0)}{\Gamma(\alpha_0 + N)} \prod_{k=1}^{K} \frac{\Gamma(\alpha_k + x_k)}{\Gamma(\alpha_k)}.$$

For parameter estimation and anomaly scoring, the combinatorial term:

$$\frac{N!}{\prod_{k=1}^{K} x_k!}$$

may be included if comparing count vectors with different total counts, because it affects the probability of the observed count vector. Some papers focus on the parameter-dependent part of the likelihood, especially when estimating $\boldsymbol{\alpha}$.

The log-likelihood contains paired differences of log-gamma terms:

$$\log \Gamma(\alpha_k + x_k) - \log \Gamma(\alpha_k),$$

$$\log \Gamma(\alpha_0 + N) - \log \Gamma(\alpha_0).$$

These paired differences are mathematically well-defined, but numerically difficult when the terms are individually huge and their difference is relatively small.

2.6 Mean-Precision and Overdispersion Parameterization

The supporting papers often reparameterize the model using:

$$\alpha_k = \frac{p_k}{\psi},$$

where:

$$\sum_{k=1}^{K} p_k = 1, \qquad \psi = \frac{1}{\alpha_0}.$$

Here:

- p_k is the mean probability of category k.
- ψ is an overdispersion parameter.
- Small ψ means large total concentration α_0, so the model approaches the Multinomial.
- Larger ψ means greater variability in the probability vector from window to window.

Different papers use slightly different conventions and thresholds for reporting dispersion. Under the mean-precision convention above, the important point is that total concentration controls how much the category probability vector can vary from window to window. Some supporting papers report a small fitted dispersion-related value and use an empirical threshold to classify datasets in their experiments. To avoid ambiguity in the thesis, the implementation should always report the exact parameter definition being used, empirical variance-to-mean diagnostics, and model-comparison results against simpler Poisson and Multinomial baselines.

2.7 Maximum Likelihood Estimation

To learn normal traffic, the system estimates $\boldsymbol{\alpha}$ from training count vectors:

$$\mathcal{D} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_R\}.$$

The maximum likelihood estimate is:

$$\hat{\boldsymbol{\alpha}} = \arg\max_{\boldsymbol{\alpha}} \sum_{i=1}^{R} \log \Pr(\mathbf{x}_i \mid \boldsymbol{\alpha}).$$

There is no closed-form solution for the Dirichlet or Dirichlet-Multinomial maximum likelihood estimate. This is why iterative methods are needed.

2.8 Digamma Function and Fixed-Point Estimation

The derivative of $\log \Gamma(x)$ is called the digamma function:

$$\psi_0(x) = \frac{d}{dx} \log \Gamma(x).$$

Minka's paper shows that the Dirichlet-Multinomial parameter vector can be estimated by a fixed-point update. For each category k, the update has the form:

$$\alpha_k^{\text{new}} = \alpha_k \frac{\sum_{i=1}^{R} [\psi_0(x_{ik} + \alpha_k) - \psi_0(\alpha_k)]}{\sum_{i=1}^{R} [\psi_0(N_i + \alpha_0) - \psi_0(\alpha_0)]},$$

where:

$$N_i = \sum_{k=1}^{K} x_{ik}, \qquad \alpha_0 = \sum_{k=1}^{K} \alpha_k.$$

Intuitively, the numerator measures how much category k is supported by the observed counts compared with the current parameter value. The denominator normalizes this support against the total count behavior of each window. Repeating this update moves $\boldsymbol{\alpha}$ toward a likelihood maximum.

Minka's work is crucial because it provides the algorithmic foundation used by the later ALM papers and by the current repository scaffold.

2.9 The Numerical Problem: Paired Log-Gamma Differences

The fixed-point update itself uses digamma differences. The convergence check and likelihood scoring use log-likelihood values. The Dirichlet-Multinomial log-likelihood requires terms like:

$$\log \Gamma(a + n) - \log \Gamma(a).$$

When a is large and n is moderate, both $\log \Gamma(a + n)$ and $\log \Gamma(a)$ may be very large, while their difference is much smaller. Subtracting two large nearly equal floating-point numbers can lose significant digits. This is called cancellation error.

This problem is not a minor implementation detail. It directly affects:

- Whether the estimator appears to converge.
- Whether two models can be compared.
- Whether anomaly scores are reliable.
- Whether the system can run on ordinary double-precision hardware.

The Yu-Shaw paper and the Languasco-Migliardi line of papers focus on this issue.

2.10 Languasco-Migliardi Log-Likelihood Computation

The Languasco-Migliardi method improves computation of paired log-gamma differences using an Euler-Maclaurin based expansion with explicit error control. The later ALM papers explain that this method:

- Avoids the instability of naive paired $\log\Gamma$ subtraction.
- Avoids the barrier encountered by the Yu-Shaw mesh approach.
- Provides a way to check whether the requested decimal precision is feasible using double precision.
- Reduces computational cost by avoiding unnecessary expensive operations.
- Can be implemented efficiently in C and wrapped for Python.

For this thesis, the LM method is not just an optimization. It is a required part of making the anomaly detector trustworthy when the data are overdispersed and likelihood values are used as security signals.

2.11 Likelihood as an Anomaly Score

Once $\hat{\boldsymbol{\alpha}}$ is estimated from normal traffic, a new window $\mathbf{x}_t$ can be scored as:

$$\text{score}(\mathbf{x}_t) = \log \Pr(\mathbf{x}_t \mid \hat{\boldsymbol{\alpha}}).$$

Because log-likelihoods are often negative, a lower value means a less likely observation.

Two practical score variants are useful:

$$\text{raw_score}(\mathbf{x}_t) = \log \Pr(\mathbf{x}_t \mid \hat{\boldsymbol{\alpha}}),$$

$$\text{normalized_score}(\mathbf{x}_t) = \frac{\log \Pr(\mathbf{x}_t \mid \hat{\boldsymbol{\alpha}})}{\max(N_t, 1)}.$$

The normalized score prevents windows with more packets from automatically dominating the score simply because they contain more observations. The thesis should evaluate both.

A threshold can be learned from validation data:

$$\text{threshold} = \text{Quantile}_q\left(\{\text{score}(\mathbf{x}_i) : \mathbf{x}_i \in \mathcal{D}_{\text{val}}\}\right).$$

For example, if the first percentile is used, then about 1 percent of normal validation windows should fall below the threshold. Any future window with:

$$\text{score}(\mathbf{x}_t) < \text{threshold}$$

is marked anomalous.

2.12 Topic Modeling Concepts as an Intuition Bridge

Dirichlet-Multinomial models are central in probabilistic topic modeling, especially Latent Dirichlet Allocation. In topic modeling:

- A document is represented as a bag of word counts.

- A topic is a probability distribution over words.
- A document has a probability distribution over topics.
- Dirichlet distributions are used as priors over probability vectors.

The IoT anomaly detection setting has a similar structure:

- A traffic window is represented as a bag of protocol or flow-feature counts.
- A normal behavior mode is a probability distribution over traffic categories.
- A device may move among behavior modes over time.
- The Dirichlet-Multinomial models variability in the category probabilities.

The proposed baseline system does not need full LDA. However, topic modeling provides a useful extension path. A future version could learn latent "traffic topics" such as idle behavior, video upload, local discovery, or update behavior. The current thesis should first build the simpler and more interpretable Dirichlet-Multinomial normal profile.

3 Detailed Literature Review

This section explains the provided sources in a way that makes their mechanisms and relevance clear without requiring the reader to open the PDFs.

3.1 Project Brief: Research Proposals Based on the Dirichlet-Multinomial

The main thesis PDF contains two research proposals. The first is a deep-learning direction called Dirichlet-Softmax. It proposes replacing or extending the classical softmax layer with a Dirichlet-Multinomial likelihood-based layer. The motivation is that ordinary softmax produces normalized class probabilities but does not represent uncertainty well, is fragile under outliers, and does not explicitly model correlations or overdispersion among categories. The proposed Dirichlet-Softmax layer would estimate $\boldsymbol{\alpha}$ parameters using an adapted Languasco-Migliardi algorithm and would target better calibrated and more informative probabilistic outputs. Suggested benchmarks include MNIST and CIFAR-10.

The second proposal is the one developed in this document. It is titled in the brief as **LM-IDNet: adaptive anomaly detection for IoT networks and cybersecurity**. Its domain is cybersecurity, IoT, and unsupervised learning. Its stated objective is to build a lightweight and efficient system for detecting anomalies in IoT networks using an unsupervised pipeline based on online estimation of Dirichlet-Multinomial parameters through the LM algorithm.

The project brief explicitly motivates the use of the Dirichlet-Multinomial by saying that traditional Gaussian and Poisson models cannot handle overdispersed data naturally. It proposes scoring new inputs by their log-likelihood under the normal model. The brief also specifies the system phases:

- Use a smart-camera traffic dataset or NetFlow data.
- Estimate an initial $\boldsymbol{\alpha}$ from normal data.
- Detect anomalies through log-likelihood.
- Support adaptive retraining.
- Target edge deployment.

It also specifies evaluation criteria:

- Accuracy.
- False positive rate.
- True positive rate.
- Log-likelihood distribution.
- Applicability on low-power devices.

These requirements define the thesis target. The system is not merely a statistical experiment; it must be a deployable pipeline with ingestion, modeling, scoring, adaptation, and evaluation.

3.2 Minka: Estimating a Dirichlet Distribution

Thomas Minka's paper, *Estimating a Dirichlet Distribution*, is the algorithmic foundation for the parameter estimation stage. It explains that the Dirichlet and Dirichlet-Multinomial distributions are basic models for proportional and count data, but their maximum likelihood estimates do not have closed-form solutions.

The paper first treats the ordinary Dirichlet distribution. It shows that the log-likelihood involves $\log\Gamma$, digamma, and trigamma functions. It then provides fixed-point and Newton-style iterative schemes. A key idea is that the Dirichlet parameters can be interpreted in terms of a mean vector and a precision parameter. The mean says where probability mass is centered; the precision says how concentrated the distribution is around that mean.

The paper then extends the discussion to the Dirichlet-Multinomial, also called the Polya distribution. It describes the Dirichlet-Multinomial as a compound process: first draw a probability vector from a Dirichlet distribution, then draw categorical outcomes from a Multinomial distribution. For count vectors, Minka derives the likelihood, its gradient, and a fixed-point update for $\boldsymbol{\alpha}$.

The most important mechanism for this thesis is the update:

$$\alpha_k^{\text{new}} = \alpha_k \frac{\sum_{i=1}^{R} \left[\psi_0(x_{ik} + \alpha_k) - \psi_0(\alpha_k)\right]}{\sum_{i=1}^{R} \left[\psi_0(N_i + \alpha_0) - \psi_0(\alpha_0)\right]}.$$

This gives a practical way to estimate a normal traffic model from count windows. The current repository already contains a Python implementation of this update in `src/algorithms/dirichlet.py`, although the broader modeling stage is not yet fully wired.

Minka also discusses Newton methods and leave-one-out approximations. These are useful as possible extensions, but the fixed-point method is the best starting point because it is conceptually simple, aligns with the ALM papers, and is already present in the codebase.

3.3 Yu and Shaw: Efficient Computation of the Dirichlet-Multinomial Log-Likelihood

Yu and Shaw's *Bioinformatics* paper addresses a numerical bottleneck: computing the Dirichlet-Multinomial log-likelihood accurately and quickly.

The paper begins from the observation that the Dirichlet-Multinomial is useful for multicategory count data with overdispersion, especially in bioinformatics settings such as microbiome data. It explains that the model reduces to the Multinomial when overdispersion approaches zero, but ordinary implementations of the log-likelihood become unstable near that limit.

The instability comes from paired log-gamma differences. A naive formula computes:

$$\log\Gamma\left(\frac{1}{\psi} + N\right) - \log\Gamma\left(\frac{1}{\psi}\right),$$

and similar category-level terms. When ψ is close to zero, $1/\psi$ is large. The two log-gamma values are huge, and subtracting them loses precision.

Yu and Shaw compare this with an alternative product formula used in some software. That product formula is more stable near zero overdispersion, but its number of terms grows with the total count N. For high-count datasets, it becomes slow.

Their proposed solution uses a truncated expansion involving Bernoulli polynomials to approximate paired log-gamma differences. Because the expansion is only directly accurate under certain conditions, they introduce a mesh algorithm. The mesh breaks the full count vector into smaller increments, computes the likelihood incrementally, and sums the pieces.

The paper demonstrates that this method improves numerical stability and runtime, including a large speed improvement on a real microbiome dataset. This is an important paper because it identifies the exact computational problem that later work improves further.

For this thesis, Yu and Shaw provide two lessons:

1. Accurate likelihood computation is essential, not optional.
2. A method can be mathematically clever but still have practical limitations if its convergence conditions or error accumulation are not fully controlled.

3.4 Languasco and Migliardi: Fast Computation of the Dirichlet-Multinomial Log-Likelihood

The Languasco-Migliardi work improves on the Yu-Shaw approach. The original paper in the resources folder, `180_2022_1311_OnlinePDF.pdf`, is repeatedly unpacked in the later ALM papers. Its contribution is a more reliable and efficient way to compute paired log-gamma differences in the Dirichlet-Multinomial log-likelihood.

The key mathematical contribution is an Euler-Maclaurin based formula for:

$$\log\Gamma\left(\frac{1}{x}+y\right)-\log\Gamma\left(\frac{1}{x}\right),$$

with an explicit remainder term. The later ALM2 paper states the theorem in operational form. It defines a correction term $d(x,y)$, a finite sum using even Bernoulli numbers, and an error term. For small cases $y=1$ or $y=2$, the expression simplifies. For larger y, the formula gives both an approximation and a bound on the approximation error.

The practical consequences are central:

- The method removes the Yu-Shaw delta-barrier, a condition that can cause Yu-Shaw to fail when some category parameters are too small relative to the selected mesh threshold.
- It estimates error accumulation rather than ignoring it.
- It allows a reliability check before computation: given the requested number of mantissa digits, the system can decide whether double precision is sufficient.
- It uses the asymptotic state:

$$A=\sum_{k=1}^{K}x_k\log p_k$$

as a way to estimate the order of magnitude of the likelihood near the low-overdispersion regime.

- It reduces computational cost by replacing expensive power, logarithm, and exponential operations where possible with repeated products.

This paper is the reason the thesis brief requires the LM algorithm. In the proposed system, the LM method should be treated as the preferred likelihood backend for convergence checks, scoring, and evaluation whenever the Dirichlet-Multinomial likelihood is needed.

3.5 Efficient Analysis of Overdispersed Data Using Accurate Dirichlet-Multinomial Computation

The TPAMI paper by Al-Haj Baddar, Languasco, and Migliardi applies the Minka fixed-point estimator while comparing two log-likelihood computation procedures: Languasco-Migliardi (LM) and Yu-Shaw (YS).

The paper begins with the modeling problem: many domains produce count data, and the correct distribution depends on dispersion. Poisson handles equidispersed univariate counts, negative binomial handles overdispersed binary or univariate count situations, and the Dirichlet-Multinomial is appropriate for multicategory overdispersed counts. The paper explicitly frames the Dirichlet-Multinomial as useful both for count modeling and as a conjugate structure in Bayesian statistics.

The paper then states the Dirichlet-Multinomial log-likelihood using p, ψ, and $\alpha_k = p_k/\psi$. It explains that evaluating this likelihood is difficult when the data are overdispersed because paired $\log\Gamma$ differences become numerically unstable.

The experimental method uses Minka's fixed-point iteration. In every iteration, the algorithm:

1. Computes digamma-difference sums from the count matrix.
2. Updates $\boldsymbol{\alpha}$.
3. Converts $\boldsymbol{\alpha}$ to $\mathbf{p}$ and ψ.
4. Compares consecutive log-likelihood values.
5. Stops when the likelihood difference is below a tolerance.

The paper compares LM and YS on multiple datasets from polls, images, and IoT network traffic. It reports that LM succeeded in estimating parameters at the required accuracy across experiments, while YS failed to produce sufficiently accurate results or failed outright in several cases. It also reports speedups for LM over YS, ranging approximately from twofold to twentyfold in the broader experiments.

The paper evaluates model quality using distances such as total variation distance, Kullback-Leibler divergence, Euclidean distance, and mean squared error. It also uses chi-square and Z-tests to evaluate whether expected and observed count vectors are sufficiently close.

For this thesis, the TPAMI paper provides evidence that:

- Minka fixed-point estimation is viable for Dirichlet-Multinomial parameter learning.
- The LM likelihood procedure is more reliable than YS for overdispersed data.
- Evaluation must include both speed and goodness-of-fit.
- IoT camera traffic is a legitimate application domain for this modeling approach.

3.6 ALM IoT-CR: Overdispersed IoT Count Modelling

The ICNC 2025 study paper is the closest source to the proposed thesis system. It applies Dirichlet-Multinomial modeling to IoT smart-camera traffic.

The paper argues that IoT security remains an arms race between attackers and defenders. Even if systems are hardened by design, defenders still need traffic models that can identify suspicious behavior and provide early warnings. A learned normal traffic profile provides a baseline for detecting unusual activity.

The dataset is the D-Link IoT device dataset. The paper focuses on `Camera5`. Packet traces are aggregated into 10-minute count vectors. Each vector contains counts for:

- TCP packets.
- UDP packets.
- SSDP packets.
- ARP packets.

The training days are packet traces from October 8, 9, 12, 13, 14, 15, and 16, 2020. The testing days are October 19, 21, and 22, 2020. This directly matches the direction already visible in the repository configuration, although the repository includes a slightly broader list of dates.

The paper uses Minka's fixed-point iteration to estimate Dirichlet-Multinomial parameters for each training sub-dataset. It compares LM and YS for modeling speed and goodness of fit. It reports that LM converges much faster. One example from the paper is especially clear: for the October 13 subset at 20 percent input size, YS required almost 8 minutes while LM required around 6 seconds. The paper reports an average speedup around 14 when excluding an extreme case, with speedups generally around one order of magnitude.

For goodness of fit, the paper uses:

- Total variation distance.
- Kullback-Leibler divergence.
- Euclidean distance.
- Mean squared error.
- Chi-square test.

- Z-test.

For this thesis, ALM IoT-CR provides a concrete experimental template:

- Use smart-camera traces.
- Aggregate to 10-minute protocol count windows.
- Use Minka fixed-point estimation.
- Compare LM, YS, and standard log-gamma.
- Evaluate speed and fit metrics.

The thesis extends this from modelling to anomaly detection, adaptive retraining, and deployment architecture.

3.7 ALM2: Fast and Accurate Implementation of the Dirichlet-Multinomial Log-Likelihood

The ACDSA 2025 implementation study paper focuses on implementation. It compares three variants of Minka's procedure:

- `MP-scipy`, using SciPy's `gammaln`.
- `MP-math`, using Python's `math.lgamma`.
- `MP-ALM`, using a C implementation of the LM technique wrapped in Python.

The paper explains that the authors originally used Python implementations of LM, then moved to C and wrapped the implementation with Python's `ctypes` module. This matters because a thesis implementation may need to run on edge devices or process many windows quickly.

The paper reports that `MP-ALM` is always faster than `MP-scipy` across the tested datasets, with gains from about 2.21x to 11.60x. Compared with `MP-math`, the gain ranges from about 1.04x to 5.02x. Larger gains are generally associated with overdispersed datasets, while smaller gains occur in non-overdispersed datasets.

The paper also explains important implementation choices:

- Avoid `pow(a, b)` for repeated powers when b is an integer-like sequence, because `pow` may internally use logarithm and exponential calls.
- Precompute normalized Bernoulli-number constants.
- Perform an error check before evaluating the main quantities.
- Use the asymptotic state to decide whether double precision can support the requested accuracy.
- Load and type-bind the C library once at runtime to minimize wrapping overhead.

For this thesis, ALM2 translates the mathematical LM method into engineering requirements. The anomaly detector should expose a likelihood engine abstraction so that the Python prototype can later be replaced by a C-backed LM implementation without changing the rest of the pipeline.

3.8 Nespoli et al.: Optimal Countermeasure Selection Against Cyber Attacks

The Nespoli, Papamartzivanos, Gomez Marmol, and Kambourakis survey is not about Dirichlet-Multinomial modeling. It is relevant because the thesis is in cybersecurity, and anomaly detection is only one phase of defense.

The survey describes a cyber-defense cycle:

1. Prevention.
2. Detection.
3. Reaction.
4. Forensics.

The proposed thesis system primarily lives in the detection phase. However, the survey explains that detection should feed reaction. Once an intrusive incident is detected, a reaction system should evaluate the impact of the event and select countermeasures. Good countermeasure selection is a multi-criteria decision problem because defenders must balance:

- Effectiveness in stopping or reducing the attack.
- Cost of the countermeasure.
- Impact on normal services.
- Time needed to deploy the response.
- Confidence in the detection.
- Risk of acting on a false positive.

The survey distinguishes static countermeasures, which are preventive, from dynamic countermeasures, which are reactive. It also discusses attack graphs, attack trees, probabilistic state models, risk assessment, and decision-support systems as tools for selecting responses.

For this thesis, the survey implies that the anomaly detector should not simply output "anomaly" or "normal." It should output enough information for a reaction layer:

- Severity score.
- Confidence.
- Affected device.
- Time window.
- Traffic categories responsible for the anomaly.
- Suggested response class.

The thesis does not need to implement a full automated countermeasure optimizer, but its architecture should be compatible with one. At minimum, it should support a human-in-the-loop response workflow.

3.9 Motivating Note on Why ALM Matters Beyond This Thesis

The text note in the resources folder explains why faster and more accurate Dirichlet-Multinomial likelihood computation matters beyond IoT security. It frames the Dirichlet-Multinomial as a core primitive in probabilistic machine learning. It appears in topic modeling, Bayesian uncertainty estimation, reinforcement learning with categorical transition uncertainty, and other models for discrete data.

The note correctly emphasizes that modern large language models and diffusion models do not usually rely on Dirichlet-Multinomial likelihoods in their core forward pass. However, improvements to this primitive still matter for:

- Topic modeling and corpus analysis.
- Hybrid neural-probabilistic systems.
- Scientific count-data modeling.
- Better uncertainty handling.
- Faster experimentation in Bayesian models.

For the thesis, this note is useful as motivation: the system is applied to IoT anomaly detection, but the computational primitive being improved is broadly relevant.

3.10 Synthesis and Research Gap

Taken together, the sources establish the following chain:

- Minka provides the iterative parameter estimation foundation.
- Yu and Shaw identify and partially solve the Dirichlet-Multinomial likelihood instability problem.
- Languasco and Migliardi improve the likelihood computation with explicit error control and better practical reliability.
- The TPAMI paper shows that LM outperforms YS across several overdispersed count datasets, including IoT traffic.
- The ALM IoT paper shows that smart-camera traffic can be modelled using this approach.
- The ALM2 paper shows how to implement the LM method efficiently through a C/Python bridge.

- The cyber-reaction survey shows that anomaly scores should feed a broader defense process.

The research gap for this Master's thesis is:

> The existing papers demonstrate accurate modelling of overdispersed IoT count data, but they do not deliver a complete adaptive anomaly detection system architecture with threshold calibration, online retraining, poisoning safeguards, interpretability, edge deployment considerations, and cybersecurity response integration.

This thesis should fill that gap.

4 Proposed System Architecture and Process Description

4.1 System Name and High-Level Concept

The proposed system is **LM-IDNet**, short for **Languasco-Migliardi IoT Dirichlet Network Detector**.

The system learns a probabilistic profile of normal IoT traffic and detects deviations through Dirichlet-Multinomial log-likelihood. It is unsupervised because it does not require labeled attacks for training. It is adaptive because it can update its normal profile over time. It is lightweight because its features are compact count vectors and its core model has only one parameter per category.

4.2 Architecture Overview

The pipeline is:

```
Raw PCAP or NetFlow
    -> ingestion and parsing
    -> categorical feature extraction
    -> fixed time-window aggregation
    -> count matrix validation
    -> overdispersion diagnostics
    -> normal-profile training
    -> LM-backed Dirichlet-Multinomial likelihood engine
    -> threshold calibration
    -> online scoring
    -> anomaly decision and explanation
    -> adaptive retraining
    -> evaluation and reporting
```

The core data object is a time-indexed count vector:

```
WindowRecord {
    device_id: string
    start_time: timestamp
    end_time: timestamp
    counts: [x_1, ..., x_K]
    total_count: N
    metadata: optional fields
}
```

The core model artifact is:

```
DMNormalModel {
    categories: [category_1, ..., category_K]
    alpha: [alpha_1, ..., alpha_K]
    alpha_0: sum(alpha)
    p_mean: alpha / alpha_0
```

```
    psi: 1 / alpha_0
    training_window_range: timestamps
    threshold_raw: float
    threshold_normalized: float
    likelihood_backend: "LM" | "standard" | "YS"
    diagnostics: convergence and fit metadata
}
```

4.3 Stage 1: Data Ingestion

The system should support two input modes.

PCAP mode: Raw packet captures are parsed packet by packet. Each packet is assigned a timestamp and a category. This mode is useful for offline research and reproducibility.

NetFlow mode: Flow summaries are used instead of raw packets. This mode is useful for deployment because many networks already export NetFlow, IPFIX, or similar records.

The current repository already contains a PCAP parser using Scapy's `PcapReader`. It identifies TCP, UDP, SSDP, and ARP packets. SSDP is recognized as UDP traffic on port 1900.

Future feature categories can include:

- DNS.
- DHCP.
- NTP.
- TLS.
- HTTP.
- ICMP.
- Destination class: local, gateway, cloud, multicast, broadcast.
- Direction: inbound, outbound.
- Packet-size bin.
- Flow-duration bin.

The thesis should begin with the four protocol categories used in the ALM IoT paper, then evaluate whether richer categories improve detection.

4.4 Stage 2: Window Aggregation

Packets are grouped into fixed windows, typically 10 minutes. For each window, the system counts category occurrences:

$$\mathbf{x}_t = \begin{bmatrix} \text{TCP}_t & \text{UDP}_t & \text{SSDP}_t & \text{ARP}_t \end{bmatrix}.$$

The ALM IoT paper uses 10-minute windows for the D-Link smart-camera dataset, so this should be the baseline. The thesis can later test sensitivity to 1-minute, 5-minute, 10-minute, and 30-minute windows.

Window length is a tradeoff:

- Short windows detect attacks faster but produce noisier count vectors.
- Long windows are more stable but delay detection and can hide short attacks.

Silent windows must be explicitly tracked. They can mean the device is idle, offline, disconnected, or not captured. A robust design should store silent windows rather than silently dropping them.

4.5 Stage 3: Data Validation and Diagnostics

Before modeling, the system validates that:

- All counts are non-negative integers.
- Category order is fixed and recorded.

- Timestamps are monotonic within each device stream.
- Missing windows are either filled with zeros or marked as missing.
- Device identity and dataset partition are recorded.

The system then computes diagnostics:

$$\bar{x}_{\cdot k} = \frac{1}{R}\sum_{i=1}^{R} x_{ik},$$

$$s_k^2 = \frac{1}{R-1}\sum_{i=1}^{R}\left(x_{ik} - \bar{x}_{\cdot k}\right)^2,$$

$$\mathrm{DI}_k = \frac{s_k^2}{\bar{x}_{\cdot k}}.$$

If $\mathrm{DI}_k > 1$, that category is overdispersed relative to a Poisson model. The system should also estimate a preliminary Dirichlet-Multinomial ψ to quantify multicategory overdispersion.

Diagnostics are not merely descriptive. They justify why the Dirichlet-Multinomial model is appropriate and provide a sanity check for the thesis.

4.6 Stage 4: Training the Normal Model

Training uses normal traffic windows. If the dataset has no labels, the initial training interval should be selected from a period believed to be benign. The D-Link camera dates used in the ALM IoT paper provide a baseline split.

Training steps:

1. Load the training count matrix:

$$X_{\text{train}} \in \mathbb{N}^{R\times K}.$$

2. Initialize $\boldsymbol{\alpha}$.

A simple initialization is:

$$\bar{x}_{\cdot k} = \frac{1}{R}\sum_{i=1}^{R} x_{ik},$$

$$p_k^{\text{init}} = \frac{\bar{x}_{\cdot k}}{\sum_{j=1}^{K}\bar{x}_{\cdot j}},$$

$$\alpha_k^{\text{init}} = c\, p_k^{\text{init}},$$

where c is an initial concentration such as 10. More robust moment-based initialization can be added later.

3. Run Minka fixed-point iteration.
4. Use the LM likelihood backend for convergence checks:

$$\left|\log L(\boldsymbol{\alpha}^{\text{new}}) - \log L(\boldsymbol{\alpha}^{\text{old}})\right| < \delta.$$

5. Persist the model artifact.

The current codebase has the fixed-point estimator in `src/algorithms/dirichlet.py`, but `src/models/modeling_stage.py` is currently commented out. A thesis implementation should activate and harden this stage.

4.7 Stage 5: Likelihood Engine

The likelihood engine should expose a common interface:

```
log_likelihood(counts, alpha, include_multinomial_coefficient=True)
```

The backend can be one of:

- Standard `gammaln`, useful as a baseline.
- Yu-Shaw, useful for literature comparison.
- Languasco-Migliardi, the target backend.
- C-wrapped LM, the optimized deployment backend.

The architecture should isolate this component because the rest of the system should not care how paired log-gamma differences are computed. This makes it possible to prototype in Python and later replace only the numerical kernel.

The likelihood engine should return:

```
value: log-likelihood
precision_status: ok | warning | impossible_requested_precision
backend_metadata: terms, error_bound, runtime
```

This is important because the LM papers emphasize reliability, not just speed.

4.8 Stage 6: Threshold Calibration

After training, the system scores a validation set of normal windows:

$$s_i = \log \Pr(\mathbf{x}_i \mid \hat{\boldsymbol{\alpha}}).$$

Threshold options:

Empirical quantile threshold:

$$\text{threshold} = \text{Quantile}_q(\{s_i\}_{i=1}^m).$$

For example, choose $q = 0.01$ for an expected 1 percent false positive rate on validation data.

Mean-standard-deviation threshold:

$$\text{threshold} = \text{mean}(\{s_i\}) - \lambda \, \text{std}(\{s_i\}).$$

This is simple but assumes roughly symmetric score variation, which may not hold.

Extreme-value threshold: Fit a tail model to low likelihood scores. This is more advanced and can be considered an extension.

The baseline thesis should use empirical quantiles because they are interpretable and do not assume Gaussian score distributions.

4.9 Stage 7: Online Anomaly Detection

At runtime, each new window is transformed into a count vector and scored:

$$\text{score}_t = \log \Pr(\mathbf{x}_t \mid \hat{\boldsymbol{\alpha}}).$$

The decision rule is:

$$\text{score}_t < \text{threshold} \quad \Longrightarrow \quad \text{anomaly}.$$

The system should output a structured event:

```
AnomalyEvent {
    device_id,
    window_start,
    window_end,
    score,
    threshold,
    severity,
    counts,
    expected_profile,
    category_contributions,
}
```

Severity can be based on the distance below threshold:

$$\text{severity} = \frac{\text{threshold} - \text{score}_t}{\text{score_scale}},$$

where score_scale may be the standard deviation or interquartile range of validation scores.

4.10 Stage 8: Explanation and Category Contributions

A useful security system must explain why it raised an alert. For each category, the system can compare observed and expected proportions:

$$\text{observed}_k = \frac{x_k}{N},$$

$$\text{expected}_k = \frac{\alpha_k}{\alpha_0},$$

$$\text{residual}_k = \text{observed}_k - \text{expected}_k \,.$$

It can also compare the observed count to an expected count:

$$\text{expected_count}_k = N \text{ expected}_k \,.$$

Large positive residuals indicate categories that are unusually high. Large negative residuals indicate categories that are unusually low.

For example:

> Anomaly: unusually high ARP and SSDP activity, unusually low TCP activity.

This helps distinguish possible causes:

- ARP burst: local network scanning or address resolution issue.
- SSDP burst: discovery abuse or device/service discovery behavior.
- TCP drop: cloud connectivity failure or device service interruption.
- UDP burst: streaming, scanning, or unusual service communication.

4.11 Stage 9: Adaptive Retraining

The project brief explicitly requires adaptive retraining. This is necessary because IoT behavior changes over time.

However, adaptation is dangerous. If the system retrains on attack traffic, it can learn the attack as normal. This is model poisoning.

The adaptive strategy should use guarded updates:

1. Maintain a buffer of recent windows classified as normal with high confidence.
2. Exclude anomalous windows and windows near the threshold.

3. Retrain only when enough safe windows are available.
4. Keep old model versions for rollback.
5. Compare new and old models before promotion.
6. Limit how quickly $\boldsymbol{\alpha}$ and thresholds can change.

A simple update policy is:

$$\text{safe_window} \quad \Longleftrightarrow \quad \text{score}_t \geq \text{threshold} + \text{margin}\,.$$

Then retrain every M safe windows or every T hours.

A more advanced policy is to maintain a sliding window of safe data and use exponential smoothing:

$$\boldsymbol{\alpha}_{\text{adapted}} = (1-\rho)\boldsymbol{\alpha}_{\text{old}} + \rho\boldsymbol{\alpha}_{\text{new}},$$

where ρ is a small adaptation rate.

4.12 Stage 11: Reaction Support

The Nespoli survey shows that detection should feed reaction. The thesis system should support at least basic response recommendations:

```
if anomaly is mild:
    log and monitor
if anomaly is repeated:
    alert operator
if anomaly is severe and category pattern suggests scan:
    recommend rate limit or isolation
if anomaly suggests discovery abuse:
    recommend SSDP filtering or local network inspection
```

The system should not automatically block traffic in the initial research prototype. Instead, it should provide structured events that could feed a later countermeasure selection framework.

A response record should include:

- Detection confidence.
- Potential impact.
- Suggested action.
- Reversibility.
- Expected operational cost.
- Whether human approval is required.

This aligns the detector with dynamic reaction frameworks without overextending the thesis.

4.13 Current Repository Mapping

The current repository already contains a useful scaffold:

- `src/processing/pcap_reader.py`: packet parsing with Scapy.
- `src/processing/transformers.py`: timestamp/protocol records to 10-minute windows.
- `src/processing/schemas.py`: Pydantic schemas for processed windows.
- `src/processing/statistics.py`: mean, variance, dispersion reporting.
- `src/algorithms/dirichlet.py`: Dirichlet-Multinomial log-likelihood with `gammaln` and Minka fixed-point estimation.
- `src/evaluation/metrics.py`: TVD, KL divergence, MSE, Brier Score, and JSD.
- `configs/d_link_day_cam5.json`: dataset paths, categories, training dates, testing dates, precision, and tolerance.

Important gaps remain:

- The modeling stage is currently commented out and must be restored.

- The CLI in tests and the CLI in `main.py` are inconsistent.
- The native C-wrapped LM backend is implemented; YS remains absent.
- No threshold calibration exists yet.
- No anomaly scoring stage exists yet.
- No adaptive retraining policy exists yet.
- No structured anomaly event output exists yet.

This is normal for a thesis scaffold. The architecture above defines how to evolve it into a research system.

4.14 Evaluation Design

The system should be evaluated along five axes.

Model fit:

- Total variation distance.
- Kullback-Leibler divergence.
- Euclidean distance.
- Mean squared error.
- Chi-square test where applicable.
- Z-test where chi-square is not applicable.

Anomaly detection:

- Accuracy.
- Precision.
- Recall / true positive rate.
- False positive rate.
- F1 score.
- ROC-AUC and PR-AUC if labeled anomalies are available.
- Detection delay.

Likelihood behavior:

- Distribution of normal log-likelihoods.
- Distribution of anomalous log-likelihoods.
- Separation between distributions.
- Stability of thresholds over time.

Efficiency and edge readiness:

- Training time.
- Scoring latency per window.
- Memory usage.
- CPU usage.
- Model size.
- Comparison of standard log-gamma, YS, Python LM, and C-wrapped LM.

4.15 Baselines

The thesis should compare the proposed model with simple baselines:

- Independent Poisson per category.
- Multinomial with fixed category probabilities.
- Gaussian model on transformed proportions.
- Standard `gammaln` Dirichlet-Multinomial.
- YS likelihood backend where feasible.
- Optional machine-learning baselines such as Isolation Forest or One-Class SVM on count/proportion features.

Baselines are essential because the thesis claim is not merely that Dirichlet-Multinomial works, but that it is better suited to overdispersed categorical IoT traffic.

5 Roadmap and Implementation Strategy

5.1 Phase 0: Research Setup and Reproducibility

Goals:

- Freeze the dataset split.
- Document the input files.
- Create a reproducible command for preprocessing, training, scoring, and evaluation.
- Fix CLI inconsistencies.

Tasks:

- Confirm available D-Link Camera5 PCAP files.
- Align `configs/d_link_day_cam5.json` with the dates used in the ALM IoT paper.
- Decide whether October 10 and 11 should remain in training or be excluded to match the paper.
- Add deterministic random seeds.
- Add experiment output directories.
- Record environment versions.

Deliverables:

- Clean configuration.
- Reproducible preprocessing run.
- Initial dataset summary report.

5.2 Phase 1: Data Processing Hardening

Goals:

- Produce validated count matrices for each day.
- Handle silent windows explicitly.
- Generate overdispersion reports.

Tasks:

- Ensure category casing is consistent across config, parser, schemas, and JSON files.
- Store window timestamps, not just counts.
- Export both JSON and matrix formats for modeling.
- Add tests for silent windows and missing categories.
- Add per-day mean, variance, dispersion index, and total-count summaries.

Deliverables:

- Processed training and testing matrices.
- Overdispersion report.
- Unit tests for processing.

5.3 Phase 2: Dirichlet-Multinomial Modeling

Goals:

- Activate model training.
- Estimate $\boldsymbol{\alpha}$, $\mathbf{p}$, and ψ.
- Save model artifacts with convergence diagnostics.

Tasks:

- Restore `run_modeling`.
- Load processed JSON windows into a count matrix.
- Implement robust alpha initialization.
- Store log-likelihood history.
- Store convergence status, iteration count, tolerance, and runtime.
- Add tests for positive alpha, monotonic or stable likelihood behavior, and model persistence.

Deliverables:

- Working `model` stage.
- Model artifact in JSON.
- Training diagnostics.

5.4 Phase 3: Likelihood Backend Abstraction

Goals:

- Separate likelihood computation from the estimator.
- Support multiple backends for comparison.

Tasks:

- Define a `LikelihoodBackend` interface.
- Implement `standard_gammaln` backend.
- Implement or integrate LM backend.
- Keep YS as optional literature comparison if feasible.
- Add numerical tests comparing backends on small cases.
- Add high-precision reference checks for selected examples if possible.

Deliverables:

- Pluggable likelihood module.
- Backend benchmark report.
- Numerical reliability tests.

5.5 Phase 4: Threshold Calibration and Anomaly Scoring

Goals:

- Convert likelihood into anomaly decisions.
- Produce structured event outputs.

Tasks:

- Split training data into fit and validation sets.
- Compute raw and normalized likelihood scores.
- Calibrate empirical quantile thresholds.
- Implement online scoring.
- Add category residual explanations.
- Export CSV/JSON anomaly events.

Deliverables:

- Working anomaly scoring stage.
- Threshold artifact.
- Likelihood distribution plots or tables.

5.6 Phase 5: Experimental Anomaly Evaluation

Goals:

- Evaluate detection performance.
- Compare against baselines.

Tasks:

- Identify any labeled attack data if available.
- If labels are not available, create controlled synthetic anomalies:
- TCP flood-like increase.
- UDP burst.
- SSDP discovery burst.
- ARP scan pattern.
- Cloud-traffic suppression.
- Mixed category drift.

- Evaluate TPR, FPR, precision, recall, and detection delay.
- Compare against Poisson, Multinomial, and optional machine-learning baselines.

Deliverables:

- Detection evaluation tables.
- Baseline comparison.
- Error analysis of false positives and false negatives.

5.7 Phase 6: Adaptive Retraining

Goals:

- Update the model under normal drift.
- Avoid poisoning from anomalous windows.

Tasks:

- Implement safe-window buffer.
- Add adaptation margin.
- Add model versioning.
- Add rollback if the new model produces worse validation behavior.
- Compare static versus adaptive models across later dates.

Deliverables:

- Adaptive retraining module.
- Static-vs-adaptive evaluation.
- Poisoning-risk discussion.

5.8 Phase 8: Edge Deployment Study

Goals:

- Demonstrate feasibility for low-power or edge environments.

Tasks:

- Measure memory and CPU use.
- Benchmark scoring latency.
- Compare pure Python and optimized likelihood backends.
- Containerize the pipeline.
- Evaluate whether the detector can run in near-real time on small windows.

Deliverables:

- Edge-readiness benchmark.
- Deployment guide.
- Recommendations for C-wrapped LM integration.

5.9 Possible Theoretical Extensions

The core thesis should remain focused, but the following extensions are natural:

Traffic-topic model: Use LDA-like latent behavior modes. Windows are documents, traffic categories are words, and topics are normal behavior modes.

Hierarchical Dirichlet-Multinomial: Learn device-level models that share information across devices while preserving device-specific behavior.

Contextual model: Condition $\boldsymbol{\alpha}$ or threshold on time of day, weekday, firmware version, or device state.

Zero-inflated model: Explicitly model silent windows or categories with many zeros.

Bayesian thresholding: Treat the threshold as uncertain and produce posterior alert probabilities.

Adversarial adaptation defense: Formalize poisoning resistance by limiting update influence and detecting drift bursts.

Dirichlet-Softmax bridge: Use the first project from the brief as a later neural extension, where model outputs represent Dirichlet concentration rather than only point probabilities.

5.10 Suggested Thesis Chapter Structure

The final thesis can be organized as:

1. Introduction and motivation.
2. Background on IoT security and anomaly detection.
3. Mathematical foundations of Dirichlet-Multinomial modeling.
4. Literature review.
5. System architecture and implementation.
6. Experimental methodology.
7. Results and analysis.
8. Adaptive retraining and deployment considerations.
9. Conclusions and future work.

5.11 Final Expected Artifacts

The project should produce:

- A working codebase for preprocessing, training, scoring, adaptation, and evaluation.
- Reproducible experiment configurations.
- Processed IoT count datasets.
- Model artifacts with alpha, psi, thresholds, and diagnostics.
- Evaluation reports.
- An architecture diagram or equivalent system description.
- A written thesis explaining both statistical theory and cybersecurity impact.

Source Map

The document integrates the provided materials as follows:

- `Proposte_di_Ricerca_basate_su_Dirichlet_Multinomial.pdf`: defines the two project options and motivates the selected LM-IDNet direction: unsupervised IoT anomaly detection through online Dirichlet-Multinomial parameter estimation, log-likelihood scoring, adaptive retraining, edge deployment, and metrics such as accuracy, false positive rate, true positive rate, and likelihood distribution.
- `minka-dirichlet.pdf`: provides the mathematical basis for estimating Dirichlet and Dirichlet-Multinomial parameters when no closed-form maximum likelihood estimator exists. The fixed-point update for $\boldsymbol{\alpha}$ is the core estimator used by the proposed system.
- `bioinformatics_30_11_1547.pdf`: explains why naive Dirichlet-Multinomial log-likelihood computation is unstable near low-overdispersion regimes and why product-form alternatives can become too slow for high-count data. Its Bernoulli-polynomial mesh method is the historical predecessor to the LM approach.
- `180_2022_1311_OnlinePDF.pdf`: contains the original Languasco-Migliardi contribution for fast paired log-gamma difference computation using an Euler-Maclaurin style expansion, explicit error control, and reliability checks for requested precision.
- `Efficient_Analysis_of_Overdispersed_Data_Using_an_Accurate_Computation_of_the_Dirichlet_Multinomial_Distribution.pdf`: shows how Minka fixed-point estimation and LM likelihood computation perform across multiple overdispersed count datasets, including IoT camera traffic, and reports LM's accuracy and speed advantages over Yu-Shaw.

- ICNC 2025 study: supplies the closest experimental template for this thesis: D-Link smart-camera traffic, 10-minute protocol count windows, training/testing day splits, LM-vs-YS modelling comparisons.
- ACDSA 2025 implementation study: translates the LM method into implementation guidance by comparing Python math, SciPy, and C-wrapped LM variants inside Minka's procedure, and by explaining performance-oriented choices such as precomputed Bernoulli constants and `ctypes` wrapping.
- `Optimal_Countermeasures_Selection_Against_Cyber_Attacks_A_Comprehensive_Survey_on_Reaction_Frameworks.pdf`: motivates the response-support layer by explaining the cyber-defense cycle and the need to balance cost, effectiveness, confidence, and service impact when selecting countermeasures.
- `WhyALMisimportant-SherenazAl-HajBaddar,[021020251.txt`: provides broader motivation for why accurate and fast Dirichlet-Multinomial likelihood computation matters beyond this particular IoT application, including topic modeling, uncertainty estimation, and hybrid probabilistic machine learning.

Concluding Statement

The proposed thesis is technically coherent because the real-world problem, data representation, statistical model, numerical method, and system architecture align tightly.

IoT anomaly detection requires lightweight unsupervised models. IoT traffic windows are naturally multicategory count vectors. These vectors are often overdispersed, so Poisson and Multinomial assumptions are too rigid. The Dirichlet-Multinomial distribution models exactly this kind of data by allowing the category probability vector to vary across windows. Minka's fixed-point algorithm gives a practical way to estimate the parameters. The Languasco-Migliardi method makes the log-likelihood computation accurate and efficient enough to trust in overdispersed regimes. The ALM IoT paper shows that the approach is already promising on smart-camera traffic. The proposed thesis completes the path from model to system by adding anomaly thresholds, adaptive retraining, interpretability, evaluation, and deployment architecture.

The result should be a defensible Master's thesis and a practical prototype: a statistically grounded, explainable, adaptive IoT anomaly detector built on a modern and numerically reliable Dirichlet-Multinomial foundation.